POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 5041 | Semester V | Theory | 4 Credits

Embedded System Design

A complete syllabus-aligned handbook with clear explanations, practical or exam guidance, Malayalam-support notes, diagrams, activities and revision tools.

Programmes	**Electronics Engineering**
Contact hours	**60**
Teaching scheme	**3:1:0:0**
Credits	**4**
CIE / ESE	**Theory CIE 40 / Theory ESE 60**
Self-learning	**5.5 h/week**

CURRICULUM INTEGRITY

Official Source Declaration and Verified Inconsistencies

The attached Revision 2026 index identifies Course 5041 as Embedded System Design in Semester V for Electronics Engineering. The official SITTTR detailed course page was checked at the indexed link and returned “Requested file not found.” With the user's approved public-source approach, this handbook is transparently based on NPTEL IIT Kharagpur’s Embedded System Design with ARM course and polytechnic embedded curricula. It does not claim to reproduce official REV2026 detailed-syllabus wording.

Source treatment: Teaching scheme, credits and assessment values are provisional placeholders aligned to neighbouring handbook format. Hardware designs, firmware code, communication settings and RTOS configurations must be based on approved engineering specifications, certified component data, current safety standards and competent professional review.

Verified source note: Verified sources support ARM architecture, embedded C, interfacing (ADC/DAC, sensors, actuators), and RTOS concepts. The four-module sequence is a learning path, not an asserted official module split.

COURSE DASHBOARD

What You Are Expected to Learn

60

Contact hours

4

Credits

Theory CIE 40

CIE

Theory ESE 60

ESE

Course introduction

Embedded System Design studies the integration of specialized hardware and software to perform dedicated functions within a larger system. It develops the ability to analyze microcontroller architectures (ARM/AVR), program in embedded C, interface with sensors and actuators, and understand Real-Time Operating Systems (RTOS) while respecting the technical and safety boundaries of embedded product development.

> **Malayalam support:** □ handbook □□□□□□□□□□□□□ syllabus topic □□□□□ □□□□□□□□□□; □□□□□□□□□ principle, diagram/procedure, application, common mistake □□□□□□ □□□□□□□□□ □□□□□□□□□.

Prerequisite foundation

Microprocessors and microcontrollers, digital electronics, C programming and basic circuit theory.

Use prerequisites only as a revision bridge. The current course outcomes and detailed syllabus define the required performance.

Teaching and assessment scheme

L	T	P	S	Self-learning/week	Contact hours	Credits	CIE	ESE
3	1	0	0	5.5 h/week	60	4	Theory CIE 40	Theory ESE 60

STUDY METHOD

How to Use This Handbook

1 · Understand

Read terms, conditions and purpose; make a short definition in your own words.

2 · Visualise

Follow the diagram, circuit, flow or procedure and label it accurately.

3 · Apply

Use the method block, calculation or practical checklist without skipping units or safety checks.

4 · Revise

Use questions, quick cards and common-mistake notes before examination.

OFFICIAL STATEMENTS

Objectives and Course Outcomes

Official course objectives

1. Explain embedded system fundamentals, classification and design metrics for dedicated applications.
2. Explain the architecture, instruction set and peripherals of a modern microcontroller (e.g., ARM Cortex-M).
3. Develop embedded C programs for interfacing sensors, actuators and communication protocols.
4. Explain the concepts and role of Real-Time Operating Systems (RTOS) in complex embedded systems.

Student-friendly meaning

You should be able to recognise the relevant system or material, explain its principle, communicate through a correct diagram/table where needed and follow a defensible solution or practical method.

> **Malayalam support:** CO □□□□□□ exam-□ □□□□□ □□□□□□□ □□□□□□ □□□□□ □□□□□□□□. Define, explain, apply □□□□□ action words □□□□□□□□□□□□.

Official Course Outcomes

CO	Official outcome	Bloom level
CO1	Explain embedded system components and design considerations for various applications.	Understand
CO2	Explain the architecture and internal peripherals of a modern microcontroller.	Understand
CO3	Develop embedded C programs for interfacing I/O devices and communication protocols.	Apply
CO4	Explain RTOS concepts including task management, scheduling and inter-process communication.	Understand

Official CO–PO articulation matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7–PO11
CO1	3	2	2	2	2	–	–
CO2	3	2	2	2	2	–	–
CO3	3	2	2	2	2	–	–
CO4	3	2	2	2	2	–	–

SYLLABUS COVERAGE

Curriculum Coverage Map

All official major topics mapped to handbook chapters

Module	Title	Official major topics	Hours	CO	Handbook section
1	Introduction to Embedded Systems	Definition and classification; components of embedded systems; design metrics (power, cost, performance); challenges in embedded design; hardware-software co-design; application areas.	Approx. 14 hours	CO1	Module 1
2	Microcontroller Architecture (ARM Focus)	Overview of ARM Cortex-M architecture; registers and memory map; instruction set basics; interrupts and exception handling; internal peripherals: GPIO, Timers, PWM, Watchdog timer.	Approx. 16 hours	CO2	Module 2
3	Embedded C and Interfacing	Embedded C programming basics; data types and pointers; interfacing with GPIO (LED, Keypad); interfacing with LCD; Analog to Digital Conversion (ADC); interfacing sensors (Temperature, LDR); communication protocols: UART, SPI, I2C.	Apply	CO3	Module 3
4	Real-Time Operating Systems (RTOS)	Concepts of RTOS; Task, States and TCB; Scheduling algorithms (Round-robin, Priority-based); Inter-process communication (IPC): Semaphores, Mutex, Mailboxes; RTOS vs OS; popular RTOS examples (FreeRTOS, uC/OS).	Approx. 14 hours	CO4	Module 4

LEARNING ROADMAP

How the Modules Connect

Introduction to Embedded Systems

Definition and classification; components of embedded systems; design metrics (power, cost, performance); challenges in embedded design; hardware-software co-design; application areas.

CO1 · Approx. 14 hours

Microcontroller Architecture (ARM Focus)

Overview of ARM Cortex-M architecture; registers and memory map; instruction set basics; interrupts and exception handling; internal peripherals: GPIO, Timers, PWM, Watchdog timer.

CO2 · Approx. 16 hours

Embedded C and Interfacing

Embedded C programming basics; data types and pointers; interfacing with GPIO (LED, Keypad); interfacing with LCD; Analog to Digital Conversion (ADC); interfacing sensors (Temperature, LDR); communication protocols: UART, SPI, I2C.

CO3 · Apply

Real-Time Operating Systems (RTOS)

Concepts of RTOS; Task, States and TCB; Scheduling algorithms (Round-robin, Priority-based); Inter-process communication (IPC): Semaphores, Mutex, Mailboxes; RTOS vs OS; popular RTOS examples (FreeRTOS, uC/OS).

CO4 · Approx. 14 hours

MODULE 1

Introduction to Embedded Systems

Definition and classification; components of embedded systems; design metrics (power, cost, performance); challenges in embedded design; hardware-software co-design; application areas.

Approx. 14 hours **CO1** **Understand · Apply**

Learning goals

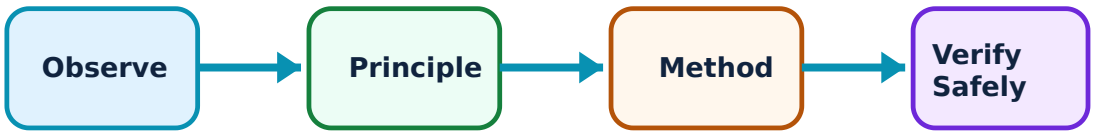
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Introduction to Embedded Systems: identify the system, state its principle, follow the correct method and verify the result safely.

1. Embedded system fundamentals–

Embedded systems are computing systems with a dedicated function within a larger mechanical or electrical system. Design metrics like power consumption, cost and real-time performance are critical. Hardware-software co-design involves the simultaneous development of both components to meet system constraints.

Study and application method

List four design metrics for an embedded system and explain their significance; compare a general-purpose computer with an embedded system.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: List four design metrics for an embedded system and explain their significance; compare a general-purpose computer with an embedded system.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Embedded systems often operate in safety-critical environments. Do not design systems for medical or automotive use without authorised safety-certification and redundant fail-safe mechanisms.

2. Hardware and software components−

Hardware includes microcontrollers, memory and I/O. Software includes firmware, device drivers and RTOS. The choice of architecture (RISC vs CISC, Harvard vs von Neumann) depends on the specific application requirements.

Study and application method

Describe the 'Harvard architecture' and explain its advantages in embedded systems; list three common embedded application areas.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Describe the 'Harvard architecture' and explain its advantages in embedded systems; list three common embedded application areas.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Improper hardware selection can lead to system instability or failure. Ensure all components meet the authorised environmental and electrical specifications of the application.

Module 1 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 2

Microcontroller Architecture (ARM Focus)

Overview of ARM Cortex-M architecture; registers and memory map; instruction set basics; interrupts and exception handling; internal peripherals: GPIO, Timers, PWM, Watchdog timer.

Approx. 16 hours **CO2** **Understand · Apply**

Learning goals

Identify core terms, explain the principle and complete the stated application or practical

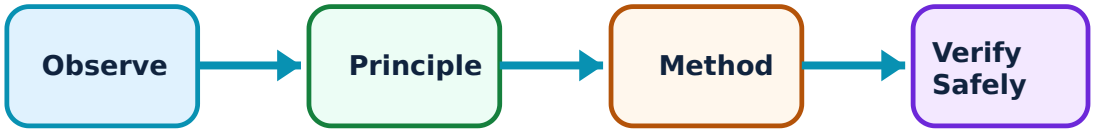
outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Microcontroller Architecture (ARM Focus): identify the system, state its principle, follow the correct method and verify the result safely.

1. ARM Cortex-M architecture−

The ARM Cortex-M is a widely used 32-bit RISC architecture for embedded systems. It features a nested vectored interrupt controller (NVIC) for low-latency interrupt handling and a standardized memory map, making it ideal for real-time control.

Study and application method

Sketch the internal block diagram of an ARM Cortex-M microcontroller; explain the role of the 'Nested Vectored Interrupt Controller' (NVIC).

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Sketch the internal block diagram of an ARM Cortex-M microcontroller; explain the role of the 'Nested Vectored Interrupt Controller' (NVIC).

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Interrupt handling must be carefully managed to prevent priority inversion or deadlocks. Use only authorised interrupt service routines (ISR) and priority settings.

2. Internal peripherals and timers−

GPIO pins allow for digital I/O. Timers are used for time delays, frequency measurement and generating Pulse Width Modulation (PWM) signals for motor control. The Watchdog timer ensures system recovery in case of software hang-ups.

Study and application method

Describe the function of a 'Watchdog Timer' in an embedded system; explain how PWM is used to control the speed of a DC motor.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Describe the function of a 'Watchdog Timer' in an embedded system; explain how PWM is used to control the speed of a DC motor.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Peripheral configuration requires precise register settings. Incorrect configuration can lead to hardware damage or incorrect system behavior. Follow authorised manufacturer reference manuals.

Module 2 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 3

Embedded C and Interfacing

Embedded C programming basics; data types and pointers; interfacing with GPIO (LED, Keypad); interfacing with LCD; Analog to Digital Conversion (ADC); interfacing sensors (Temperature, LDR); communication protocols: UART, SPI, I2C.

Apply | CO3 | Understand · Apply

Learning goals

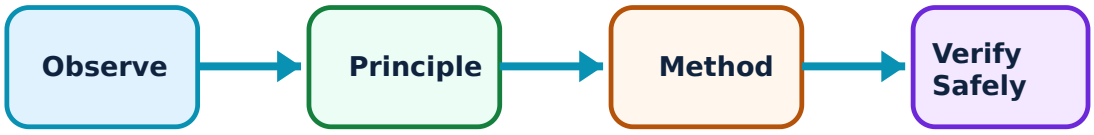
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Embedded C and Interfacing: identify the system, state its principle, follow the correct method and verify the result safely.

1. Embedded C and GPIO interfacing−

Embedded C extends C with hardware-specific features. GPIO interfacing involves configuring pins as input or output and reading/writing their states. Keypads and LCDs are standard user interface components for embedded devices.

Study and application method

Write a conceptual C code snippet to blink an LED connected to a GPIO pin; explain the importance of 'volatile' keyword in embedded C.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Write a conceptual C code snippet to blink an LED connected to a GPIO pin; explain the importance of 'volatile' keyword in embedded C.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: GPIO pins have current limits. Do not connect high-power loads directly to GPIO pins without authorised isolation or driver circuits (e.g., transistors/relays).

2. ADC and communication protocols–

ADC converts analog sensor signals (e.g., from temperature sensors) into digital data. Serial communication protocols like UART (point-to-point), I2C (multi-master) and SPI (high-speed) allow the microcontroller to interact with other chips and modules.

Study and application method

Compare UART, I2C and SPI protocols in a conceptual table based on speed and number of wires; describe the process of ADC sampling.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Compare UART, I2C and SPI protocols in a conceptual table based on speed and number of wires; describe the process of ADC sampling.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Communication protocols require precise timing and voltage levels. Ensure all bus pull-ups and signal terminations follow authorised engineering standards to prevent data corruption.

Module 3 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 4

Real-Time Operating Systems (RTOS)

Concepts of RTOS; Task, States and TCB; Scheduling algorithms (Round-robin, Priority-based); Inter-process communication (IPC): Semaphores, Mutex, Mailboxes; RTOS vs OS; popular RTOS examples (FreeRTOS, uC/OS).

Approx. 14 hours **CO4** **Understand · Apply**

Learning goals

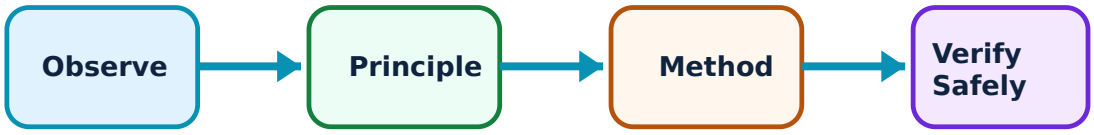
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Real-Time Operating Systems (RTOS): identify the system, state its principle, follow the correct method and verify the result safely.

1. RTOS task management and scheduling–

An RTOS manages multiple tasks (threads) with deterministic timing. Tasks move through states like Ready, Running and Blocked. The scheduler decides which task runs based on priority or time-slicing to meet real-time deadlines.

Study and application method

Define a 'Task Control Block' (TCB); compare 'Preemptive' and 'Non-preemptive' scheduling conceptually.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Define a 'Task Control Block' (TCB); compare 'Preemptive' and 'Non-preemptive' scheduling conceptually.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Real-time deadlines are absolute. Do not implement RTOS-based control for critical systems without authorised worst-case execution time (WCET) analysis.

2. IPC and synchronization−

Inter-process communication (IPC) allows tasks to share data and synchronize. Semaphores and Mutexes prevent race conditions when accessing shared resources. Mailboxes and Queues allow for message passing between tasks.

Study and application method

Explain the difference between a 'Semaphore' and a 'Mutex'; describe the concept of 'Priority Inversion' and how it can be mitigated.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Explain the difference between a 'Semaphore' and a 'Mutex'; describe the concept of 'Priority Inversion' and how it can be mitigated.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Incorrect use of IPC can lead to deadlocks or unpredictable system behavior. Use only authorised synchronization primitives and validated RTOS configurations.

Module 4 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

FORMULA AND PROCEDURE BANK

Rapid Recall Bank

Recall 1

State symbols and SI units before substitution.

Recall 2

Draw the circuit, system boundary, vector or process flow before reasoning.

Recall 3

> For laboratory work: aim → apparatus → diagram → procedure → observation → calculation → result → precautions.

Recall 4

> For a graph: label axes, units, scale, operating region and conclusion.

Recall 5

> For a comparison: use construction, working, result, advantage and limitation.

Recall 6

> For an extended answer: definition/principle, labelled diagram, working steps, application and a caution/limitation.

DIAGRAM LIBRARY

Diagram Library for Rapid Revision

Module 1: Introduction to Embedded Systems

Use the concept flow to revise observation, principle, method and verification.

Module 2: Microcontroller Architecture (ARM Focus)

Use the concept flow to revise observation, principle, method and verification.

Module 3: Embedded C and Interfacing

Use the concept flow to revise observation, principle, method and verification.

Module 4: Real-Time Operating Systems (RTOS)

Use the concept flow to revise observation, principle, method and verification.

SELF-LEARNING

Official Self-Learning and Student Activities

Structured activity

Compare ARM and 8051 architectures in a conceptual table showing three differences.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Sketch the interfacing diagram of a 16x2 LCD with a microcontroller.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Write a conceptual C function to read data from an I2C temperature sensor.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Draw a task state transition diagram for a typical RTOS.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Identify the communication protocols used in a modern smartphone or a smart home device.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Activity discipline: The supplied PDF assigns the listed self-learning hours. This handbook preserves the activity direction; the teacher’s approved schedule and safety instructions govern execution.

EXAMINATION READINESS

Assessment Methodology and Examination Pattern

Official assessment structure

Component	Official mark / conversion	Student evidence
Attendance	5 marks	End-of-semester attendance component.
Self-learning activities	15 marks	Module-linked activities.
Series examinations	20 + 20 converted	Two 50-mark series examinations.
CIE total	Theory CIE 40	Continuous internal evaluation.
Written ESE	Theory ESE 60	2.5 hours; official Part A / Part B / Part C pattern.

Series-test pattern

The supplied theory pattern uses Part A (1-mark), Part B (3-mark with choice) and Part C (7-mark) distribution across the stated modules. Practical courses follow the supplied practical-test scheme.

Examination evidence

Show a complete method, correct units/conditions, clear diagrams or tables, a result/conclusion and safety awareness for any apparatus or process involved.

EXAM PRACTICE

Module-wise Questions with Answers

Module 1 — Introduction to Embedded Systems

Explain Embedded system fundamentals and state one correct method, application or precaution.–

Embedded systems are computing systems with a dedicated function within a larger mechanical or electrical system. Design metrics like power consumption, cost and real-time performance are critical. Hardware-software co-design involves the simultaneous development of both components to meet system constraints.

Answer framework: List four design metrics for an embedded system and explain their significance; compare a general-purpose computer with an embedded system.

Important point: Embedded systems often operate in safety-critical environments. Do not design systems for medical or automotive use without authorised safety-certification and redundant fail-safe mechanisms.

Explain Hardware and software components and state one correct method, application or precaution.–

Hardware includes microcontrollers, memory and I/O. Software includes firmware, device drivers and RTOS. The choice of architecture (RISC vs CISC, Harvard vs von Neumann) depends on the specific application requirements.

Answer framework: Describe the 'Harvard architecture' and explain its advantages in embedded systems; list three common embedded application areas.

Important point: Improper hardware selection can lead to system instability or failure. Ensure all components meet the authorised environmental and electrical specifications of the application.

Module 2 — Microcontroller Architecture (ARM Focus)

Explain ARM Cortex-M architecture and state one correct method, application or precaution.–

The ARM Cortex-M is a widely used 32-bit RISC architecture for embedded systems. It features a nested vectored interrupt controller (NVIC) for low-latency interrupt handling and a standardized memory map, making it ideal for real-time control.

Answer framework: Sketch the internal block diagram of an ARM Cortex-M microcontroller; explain the role of the 'Nested Vectored Interrupt Controller' (NVIC).

Important point: Interrupt handling must be carefully managed to prevent priority inversion or deadlocks. Use only authorised interrupt service routines (ISR) and priority settings.

Explain Internal peripherals and timers and state one correct method, application or precaution.–

GPIO pins allow for digital I/O. Timers are used for time delays, frequency measurement and generating Pulse Width Modulation (PWM) signals for motor control. The Watchdog timer ensures system recovery in case of software hang-ups.

Answer framework: Describe the function of a 'Watchdog Timer' in an embedded system; explain how PWM is used to control the speed of a DC motor.

Important point: Peripheral configuration requires precise register settings. Incorrect configuration can lead to hardware damage or incorrect system behavior. Follow authorised manufacturer reference manuals.

Module 3 — Embedded C and Interfacing

Explain Embedded C and GPIO interfacing and state one correct method, application or precaution.–

Embedded C extends C with hardware-specific features. GPIO interfacing involves configuring pins as input or output and reading/writing their states. Keypads and LCDs are standard user interface components for embedded devices.

Answer framework: Write a conceptual C code snippet to blink an LED connected to a GPIO pin; explain the importance of 'volatile' keyword in embedded C.

Important point: GPIO pins have current limits. Do not connect high-power loads directly to GPIO pins without authorised isolation or driver circuits (e.g., transistors/relays).

Explain ADC and communication protocols and state one correct method, application or precaution.–

ADC converts analog sensor signals (e.g., from temperature sensors) into digital data. Serial communication protocols like UART (point-to-point), I2C (multi-master) and SPI (high-speed) allow the microcontroller to interact with other chips and modules.

Answer framework: Compare UART, I2C and SPI protocols in a conceptual table based on speed and number of wires; describe the process of ADC sampling.

Important point: Communication protocols require precise timing and voltage levels. Ensure all bus pull-ups and signal terminations follow authorised engineering standards to prevent data corruption.

Module 4 — Real-Time Operating Systems (RTOS)

Explain RTOS task management and scheduling and state one correct method, application or precaution.–

An RTOS manages multiple tasks (threads) with deterministic timing. Tasks move through states like Ready, Running and Blocked. The scheduler decides which task runs based on priority or time-slicing to meet real-time deadlines.

Answer framework: Define a 'Task Control Block' (TCB); compare 'Preemptive' and 'Non-preemptive' scheduling conceptually.

Important point: Real-time deadlines are absolute. Do not implement RTOS-based control for critical systems without authorised worst-case execution time (WCET) analysis.

Explain IPC and synchronization and state one correct method, application or precaution.–

Inter-process communication (IPC) allows tasks to share data and synchronize. Semaphores and Mutexes prevent race conditions when accessing shared resources. Mailboxes and Queues allow for message passing between tasks.

Answer framework: Explain the difference between a 'Semaphore' and a 'Mutex'; describe the concept of 'Priority Inversion' and how it can be mitigated.

Important point: Incorrect use of IPC can lead to deadlocks or unpredictable system behavior. Use only authorised synchronization primitives and validated RTOS configurations.

PRACTICE ONLY

Practice Model Paper

Important: This practice paper is based on the official pattern in the supplied syllabus. It is **not** an official SITTTR model question paper.

Part A — short response

1. State one key definition from Module 1: Introduction to Embedded Systems.
2. Identify one diagram, observation, formula or safety condition relevant to Module 1.
3. State one key definition from Module 2: Microcontroller Architecture (ARM Focus).
4. Identify one diagram, observation, formula or safety condition relevant to Module 2.
5. State one key definition from Module 3: Embedded C and Interfacing.
6. Identify one diagram, observation, formula or safety condition relevant to Module 3.
7. State one key definition from Module 4: Real-Time Operating Systems (RTOS).

8. Identify one diagram, observation, formula or safety condition relevant to Module 4.

Part B — structured explanation

1. Explain a selected procedure/principle from Module 1 with a labelled diagram, table or method.
2. Explain a selected procedure/principle from Module 2 with a labelled diagram, table or method.
3. Explain a selected procedure/principle from Module 3 with a labelled diagram, table or method.
4. Explain a selected procedure/principle from Module 4 with a labelled diagram, table or method.

Part C — extended response / practical task

1. Develop a complete answer or practical plan for: Definition and classification; components of embedded systems; design metrics (power, cost, performance); challenges in embedded design; hardware-software co-design; application areas..
2. Develop a complete answer or practical plan for: Overview of ARM Cortex-M architecture; registers and memory map; instruction set basics; interrupts and exception handling; internal peripherals: GPIO, Timers, PWM, Watchdog timer..
3. Develop a complete answer or practical plan for: Embedded C programming basics; data types and pointers; interfacing with GPIO (LED, Keypad); interfacing with LCD; Analog to Digital Conversion (ADC); interfacing sensors (Temperature, LDR); communication protocols: UART, SPI, I2C..
4. Develop a complete answer or practical plan for: Concepts of RTOS; Task, States and TCB; Scheduling algorithms (Round-robin, Priority-based); Inter-process communication (IPC): Semaphores, Mutex, Mailboxes; RTOS vs OS; popular RTOS examples (FreeRTOS, uC/OS)..

LAST-MILE REVISION

Quick Revision

Module 1: Introduction to Embedded Systems

Definition and classification; components of embedded systems; design metrics (power, cost, performance); challenges in embedded design; hardware-software co-design; application areas.

- Match each topic to CO1.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 2: Microcontroller Architecture (ARM Focus)

Overview of ARM Cortex-M architecture; registers and memory map; instruction set basics; interrupts and exception handling; internal peripherals: GPIO, Timers, PWM, Watchdog timer.

- Match each topic to CO2.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 3: Embedded C and Interfacing

Embedded C programming basics; data types and pointers; interfacing with GPIO (LED, Keypad); interfacing with LCD; Analog to Digital Conversion (ADC); interfacing sensors (Temperature, LDR); communication protocols: UART, SPI, I2C.

- Match each topic to CO3.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 4: Real-Time Operating Systems (RTOS)

Concepts of RTOS; Task, States and TCB; Scheduling algorithms (Round-robin, Priority-based); Inter-process communication (IPC): Semaphores, Mutex, Mailboxes; RTOS vs OS; popular RTOS examples (FreeRTOS, uC/OS).

- Match each topic to CO4.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Common mistakes: Writing an unlabelled diagram; ignoring units, polarity or conditions; treating a practical result as valid without observations; confusing a definition with an application; and omitting a safety precaution where the topic uses apparatus, current, heat, chemicals or tools.

Exam checklist: Read the command word; answer every part; draw clean labelled diagrams; use a clear calculation/procedure; state result with units or observation; and reserve two minutes for checking.

VOCABULARY

Glossary

Important terms from the syllabus	
Term / topic	**One-line meaning**
Embedded system fundamentals	Embedded systems are computing systems with a dedicated function within a larger mechanical or electrical system.
Hardware and software components	Hardware includes microcontrollers, memory and I/O.
ARM Cortex-M architecture	The ARM Cortex-M is a widely used 32-bit RISC architecture for embedded systems.
Internal peripherals and timers	GPIO pins allow for digital I/O.
Embedded C and GPIO interfacing	Embedded C extends C with hardware-specific features.
ADC and communication protocols	ADC converts analog sensor signals (e.
RTOS task management and scheduling	An RTOS manages multiple tasks (threads) with deterministic timing.
IPC and synchronization	Inter-process communication (IPC) allows tasks to share data and synchronize.

LEARNING RESOURCES

References

Recommended embedded-design references

1. Indranil Sengupta and Kamalika Datta, Embedded System Design with ARM.
2. Shibu K. V., Introduction to Embedded Systems.
3. Raj Kamal, Embedded Systems: Architecture, Programming and Design.
4. Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors.

Approved public embedded-design sources

- https://nptel.ac.in/courses/106105193
- https://onlinecourses.nptel.ac.in/noc20_ee98/preview

These sources provide the verified study basis while official Course 5041 detail is unavailable; they do not replace official hardware manuals, authorised firmware code, certified designs or authorised industrial automation plans.